
Evolución del Software Dev-Ops

INFO 248

Dra. Valeria Henríquez

Evolución del software

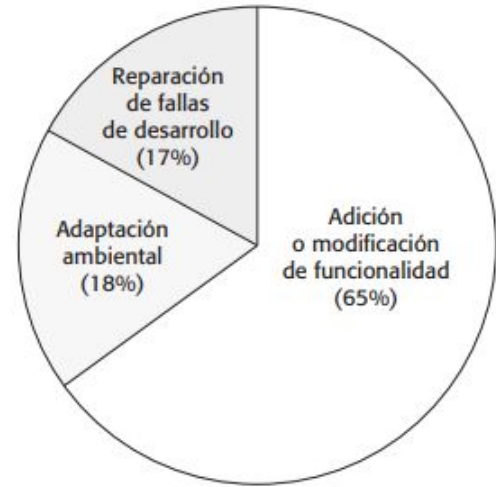
El desarrollo del software **no se detiene cuando un sistema se entrega**, sino que continúa a lo largo de su vida. Después de distribuir un sistema, inevitablemente debe modificarse, con la finalidad de mantenerlo útil.

Evolución del software

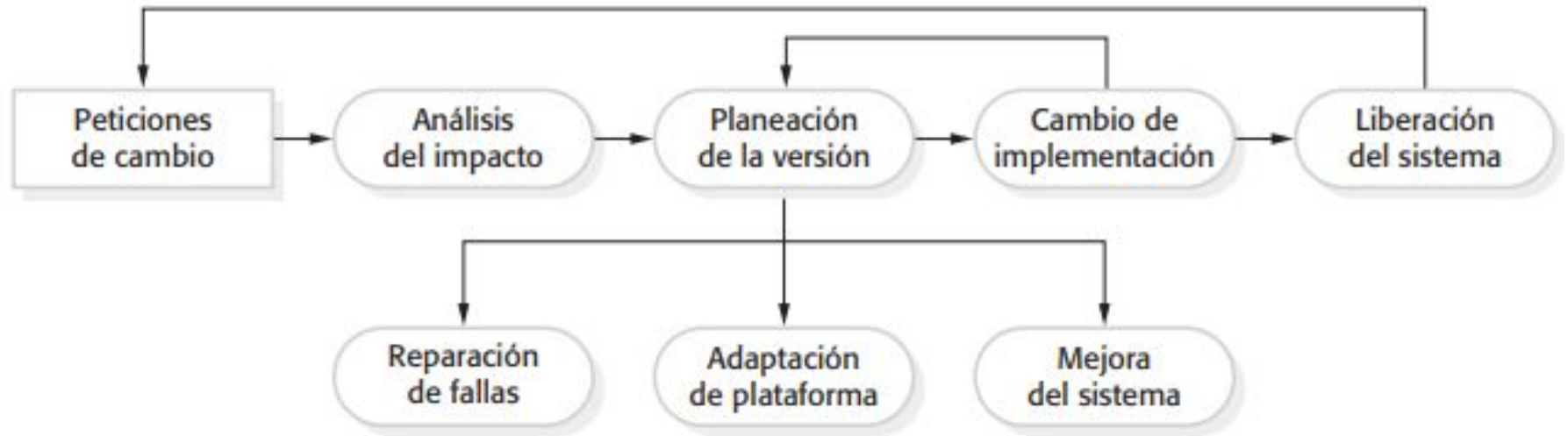
- Erlikh (2000) sugiere que entre el 85 y 90% de los costos del software organizacional son costos de evolución.
- Otros estudios sugieren que éstos conforman alrededor de dos tercios de los costos del software.
- Los costos del cambio del software representan una gran parte del presupuesto de TI de todas las compañías.

Mantenimiento del software

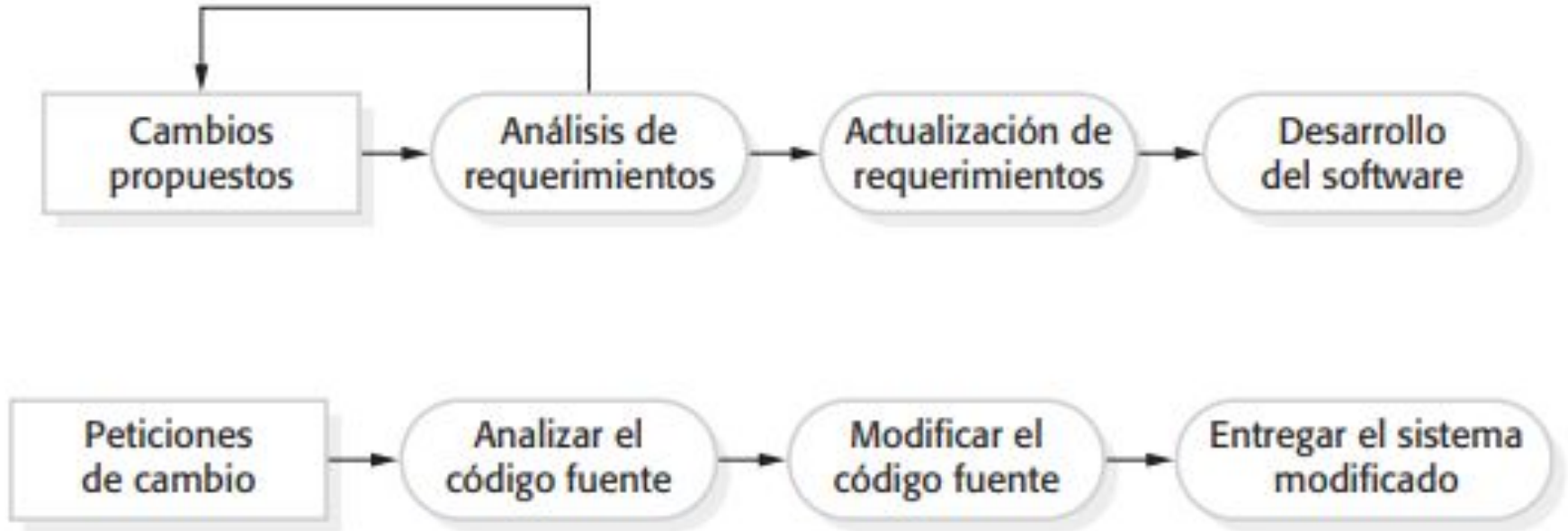
1. **Reparaciones de fallas.** Los errores de codificación por lo general son relativamente baratos de corregir; los errores de diseño son más costosos, ya que quizás impliquen la reescritura de muchos componentes del programa. Los errores de requerimientos son los más costosos de reparar debido a que podría ser necesario un extenso rediseño del sistema.
2. **Adaptación ambiental.** Este tipo de mantenimiento se requiere cuando algún aspecto del entorno del sistema, como el hardware, la plataforma operativa del sistema u otro soporte, cambia el software. El sistema de aplicación tiene que modificarse para lidiar con dichos cambios ambientales.
3. **Adición de funcionalidad.** Este tipo de mantenimiento es necesario cuando varían los requerimientos del sistema, en respuesta a un cambio organizacional o empresarial. La escala de los cambios requeridos en el software suele ser mucho mayor que en los otros tipos de mantenimiento



Proceso de evolución del software



Proceso de Implementación del cambio vs reparación de emergencia



“Leyes de Lehman” relacionadas al cambio del sistema

Lehman y Belady realizaron varios estudios empíricos acerca del cambio al sistema, para entender más sobre las características de la evolución del software.

Lehman y Belady afirman que dichas leyes suelen ser verdaderas para todos los tipos de sistemas de software organizacional grandes.

El mantenimiento del sistema es un proceso **inevitable**. A medida que cambia el entorno del sistema, surgen nuevos requerimientos y el sistema debe modificarse. Cuando el sistema modificado se reintroduce al entorno, promueve más cambios ambientales, de manera que el proceso de evolución comienza de nuevo.

Ley	Descripción
Cambio continuo	Un programa usado en un entorno real debe cambiar; de otro modo, en dicho entorno se volvería progresivamente inútil.
Complejidad creciente	A medida que cambia un programa en evolución, su estructura tiende a volverse más compleja. Deben dedicarse recursos adicionales para conservar y simplificar su estructura.
Evolución de programa grande	La evolución del programa es un proceso autorregulador. Los atributos del sistema, como tamaño, tiempo entre versiones y número de errores reportados, son casi invariantes para cada versión del sistema.
Estabilidad organizacional	Durante la vida de un programa, su tasa de desarrollo es aproximadamente constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de familiaridad	A lo largo de la existencia de un sistema, el cambio incremental en cada liberación es casi constante.
Crecimiento continuo	La funcionalidad ofrecida por los sistemas tiene que aumentar continuamente para mantener la satisfacción del usuario.
Declive de calidad	La calidad de los sistemas declinará, a menos que se modifiquen para reflejar los cambios en su entorno operacional.
Sistema de retroalimentación	Los procesos de evolución incorporan sistemas de retroalimentación multiagente y multiciclo. Además, deben tratarse como sistemas de retroalimentación para lograr una mejora significativa del producto.

Conforme cambia un sistema, su **estructura se degrada**. La única manera de evitar que esto ocurra es invertir en **mantenimiento preventivo**. Se invierte tiempo mejorando la estructura del software sin agregar nada a su funcionalidad. Evidentemente, esto significa costos adicionales, por encima de los asignados para implementar los cambios requeridos al sistema.

Ley	Descripción
Cambio continuo	Un programa usado en un entorno real debe cambiar; de otro modo, en dicho entorno se volvería progresivamente inútil.
Complejidad creciente	A medida que cambia un programa en evolución, su estructura tiende a volverse más compleja. Deben dedicarse recursos adicionales para conservar y simplificar su estructura.
Evolución de programa grande	La evolución del programa es un proceso autorregulador. Los atributos del sistema, como tamaño, tiempo entre versiones y número de errores reportados, son casi invariantes para cada versión del sistema.
Estabilidad organizacional	Durante la vida de un programa, su tasa de desarrollo es aproximadamente constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de familiaridad	A lo largo de la existencia de un sistema, el cambio incremental en cada liberación es casi constante.
Crecimiento continuo	La funcionalidad ofrecida por los sistemas tiene que aumentar continuamente para mantener la satisfacción del usuario.
Declive de calidad	La calidad de los sistemas declinará, a menos que se modifiquen para reflejar los cambios en su entorno operacional.
Sistema de retroalimentación	Los procesos de evolución incorporan sistemas de retroalimentación multiagente y multiciclo. Además, deben tratarse como sistemas de retroalimentación para lograr una mejora significativa del producto.

Los sistemas grandes tienen una **dinámica propia que se establece en una etapa temprana del proceso de desarrollo**. Esto determina las grandes tendencias del proceso de mantenimiento del sistema y limita el número de cambios posibles al sistema.

(Una ley Polémica)

Ley	Descripción
Cambio continuo	Un programa usado en un entorno real debe cambiar; de otro modo, en dicho entorno se volvería progresivamente inútil.
Complejidad creciente	A medida que cambia un programa en evolución, su estructura tiende a volverse más compleja. Deben dedicarse recursos adicionales para conservar y simplificar su estructura.
Evolución de programa grande	La evolución del programa es un proceso autorregulador. Los atributos del sistema, como tamaño, tiempo entre versiones y número de errores reportados, son casi invariantes para cada versión del sistema.
Estabilidad organizacional	Durante la vida de un programa, su tasa de desarrollo es aproximadamente constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de familiaridad	A lo largo de la existencia de un sistema, el cambio incremental en cada liberación es casi constante.
Crecimiento continuo	La funcionalidad ofrecida por los sistemas tiene que aumentar continuamente para mantener la satisfacción del usuario.
Declive de calidad	La calidad de los sistemas declinará, a menos que se modifiquen para reflejar los cambios en su entorno operacional.
Sistema de retroalimentación	Los procesos de evolución incorporan sistemas de retroalimentación multiagente y multiciclo. Además, deben tratarse como sistemas de retroalimentación para lograr una mejora significativa del producto.

Ley	Descripción
Cambio continuo	Un programa usado en un entorno real debe cambiar; de otro modo, en dicho entorno se volvería progresivamente inútil.
Complejidad creciente	A medida que cambia un programa en evolución, su estructura tiende a volverse más compleja. Deben dedicarse recursos adicionales para conservar y simplificar su estructura.
Evolución de programa grande	La evolución del programa es un proceso autorregulador. Los atributos del sistema, como tamaño, tiempo entre versiones y número de errores reportados, son casi invariantes para cada versión del sistema.
Estabilidad organizacional	Durante la vida de un programa, su tasa de desarrollo es aproximadamente constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de familiaridad	A lo largo de la existencia de un sistema, el cambio incremental en cada liberación es casi constante.
Crecimiento continuo	La funcionalidad ofrecida por los sistemas tiene que aumentar continuamente para mantener la satisfacción del usuario.
Declive de calidad	La calidad de los sistemas declinará, a menos que se modifiquen para reflejar los cambios en su entorno operacional.
Sistema de retroalimentación	Los procesos de evolución incorporan sistemas de retroalimentación multiagente y multiciclo. Además, deben tratarse como sistemas de retroalimentación para lograr una mejora significativa del producto.

La mayoría de los grandes proyectos de programación funcionan en un estado "saturado". Es decir, **un cambio en los recursos o en el personal tiene efectos imperceptibles en la evolución a largo plazo del sistema.**

(Una ley Polémica)

Ley	Descripción
Cambio continuo	Un programa usado en un entorno real debe cambiar; de otro modo, en dicho entorno se volvería progresivamente inútil.
Complejidad creciente	A medida que cambia un programa en evolución, su estructura tiende a volverse más compleja. Deben dedicarse recursos adicionales para conservar y simplificar su estructura.
Evolución de programa grande	La evolución del programa es un proceso autorregulador. Los atributos del sistema, como tamaño, tiempo entre versiones y número de errores reportados, son casi invariantes para cada versión del sistema.
Estabilidad organizacional	Durante la vida de un programa, su tasa de desarrollo es aproximadamente constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de familiaridad	A lo largo de la existencia de un sistema, el cambio incremental en cada liberación es casi constante.
Crecimiento continuo	La funcionalidad ofrecida por los sistemas tiene que aumentar continuamente para mantener la satisfacción del usuario.
Declive de calidad	La calidad de los sistemas declinará, a menos que se modifiquen para reflejar los cambios en su entorno operacional.
Sistema de retroalimentación	Los procesos de evolución incorporan sistemas de retroalimentación multiagente y multiciclo. Además, deben tratarse como sistemas de retroalimentación para lograr una mejora significativa del producto.

Agregar nueva funcionalidad a un sistema introduce inevitablemente nuevas fallas al mismo. **Cuanto más funcionalidad se agregue en cada versión, más fallas habrá.**

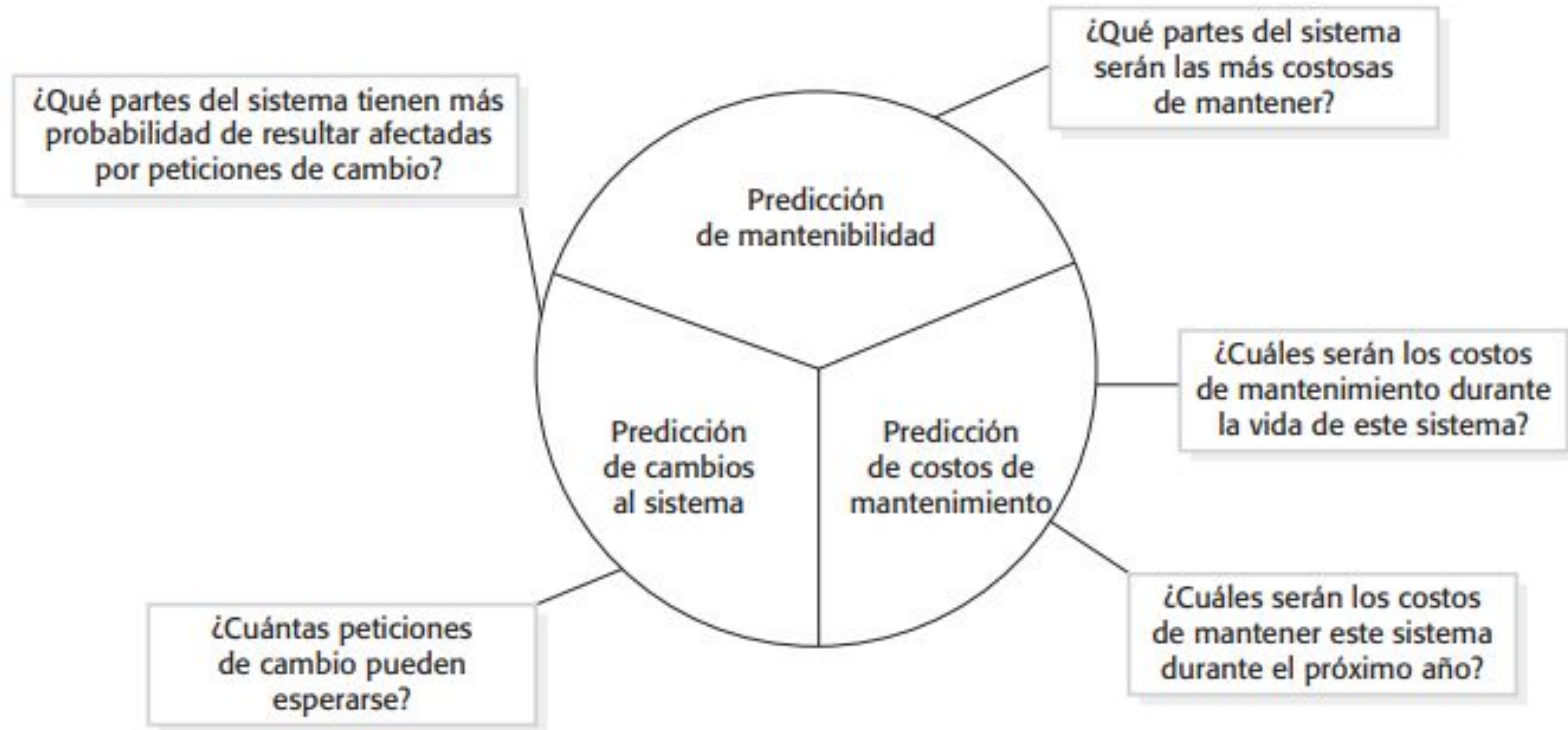
Ley	Descripción
Cambio continuo	Un programa usado en un entorno real debe cambiar; de otro modo, en dicho entorno se volvería progresivamente inútil.
Complejidad creciente	A medida que cambia un programa en evolución, su estructura tiende a volverse más compleja. Deben dedicarse recursos adicionales para conservar y simplificar su estructura.
Evolución de programa grande	La evolución del programa es un proceso autorregulador. Los atributos del sistema, como tamaño, tiempo entre versiones y número de errores reportados, son casi invariantes para cada versión del sistema.
Estabilidad organizacional	Durante la vida de un programa, su tasa de desarrollo es aproximadamente constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de familiaridad	A lo largo de la existencia de un sistema, el cambio incremental en cada liberación es casi constante.
Crecimiento continuo	La funcionalidad ofrecida por los sistemas tiene que aumentar continuamente para mantener la satisfacción del usuario.
Declive de calidad	La calidad de los sistemas declinará, a menos que se modifiquen para reflejar los cambios en su entorno operacional.
Sistema de retroalimentación	Los procesos de evolución incorporan sistemas de retroalimentación multiagente y multiciclo. Además, deben tratarse como sistemas de retroalimentación para lograr una mejora significativa del producto.

Ambas leyes apuntan a que los usuarios de software se volverán cada vez más infortunados con el sistema, a menos que se le mantenga y se le agregue nueva funcionalidad

Ley	Descripción
Cambio continuo	Un programa usado en un entorno real debe cambiar; de otro modo, en dicho entorno se volvería progresivamente inútil.
Complejidad creciente	A medida que cambia un programa en evolución, su estructura tiende a volverse más compleja. Deben dedicarse recursos adicionales para conservar y simplificar su estructura.
Evolución de programa grande	La evolución del programa es un proceso autorregulador. Los atributos del sistema, como tamaño, tiempo entre versiones y número de errores reportados, son casi invariantes para cada versión del sistema.
Estabilidad organizacional	Durante la vida de un programa, su tasa de desarrollo es aproximadamente constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de familiaridad	A lo largo de la existencia de un sistema, el cambio incremental en cada liberación es casi constante.
Crecimiento continuo	La funcionalidad ofrecida por los sistemas tiene que aumentar continuamente para mantener la satisfacción del usuario.
Declive de calidad	La calidad de los sistemas declinará, a menos que se modifiquen para reflejar los cambios en su entorno operacional.
Sistema de retroalimentación	Los procesos de evolución incorporan sistemas de retroalimentación multiagente y multiciclo. Además, deben tratarse como sistemas de retroalimentación para lograr una mejora significativa del producto.

No existe una aplicación práctica. Se debe entender el proceso evolutivo como un sistema con retroalimentación.

Predicción de mantenimiento

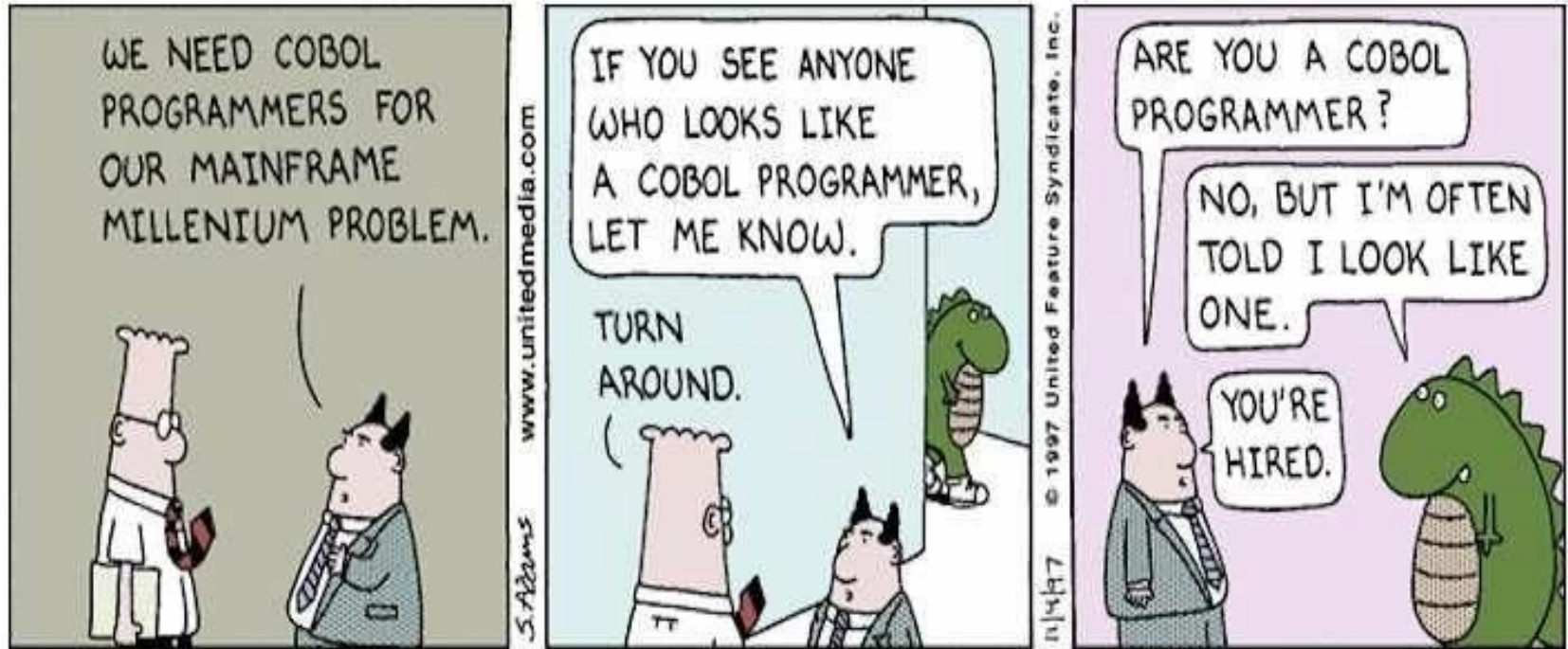


Sistemas heredados/legados/legacy

Los sistemas heredados son **sistemas antiguos que todavía son útiles y en ocasiones críticos para la operación** de la empresa. Pueden implementarse usando **lenguajes y tecnología obsoletos**, o utilizar otros sistemas que sean costosos de mantener. Normalmente su estructura se ha degradado por cambios y **la documentación está extraviada o desactualizada**.

No obstante, quizá **no sea efectivo en costo sustituir tales sistemas**. Quizá sólo se usen algunas veces al año o sea demasiado riesgoso sustituirlos porque se hayan perdido las especificaciones.

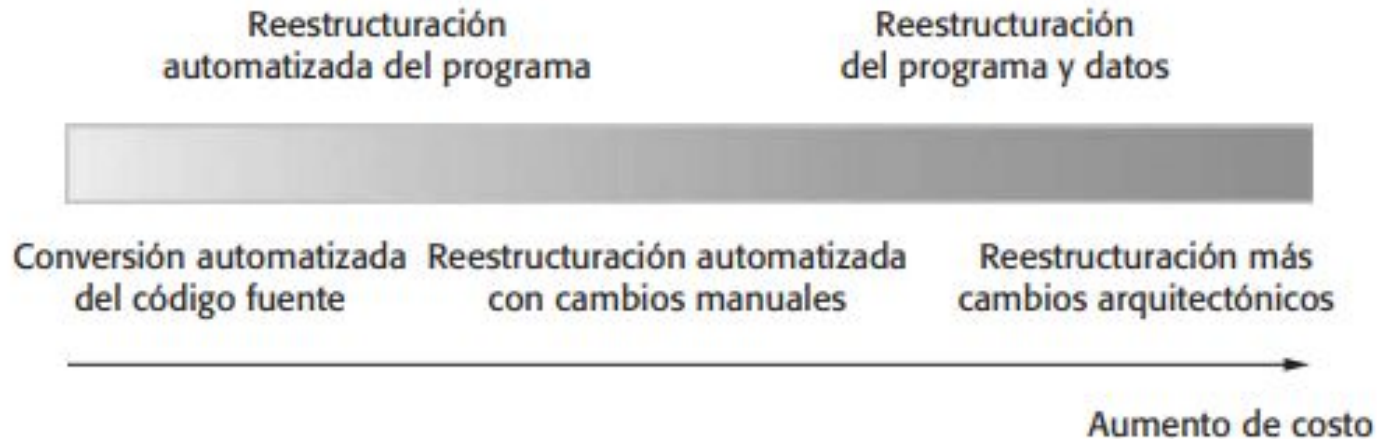
COBOL: Completely Obsolete But Omnipresent Language



El proceso de reingeniería



El proceso de reingeniería



¿Qué debemos hacer para evitar/reducir estos costos?

Mantenimiento preventivo mediante refactorización

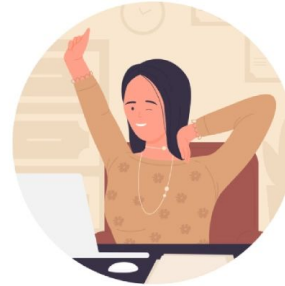
Código duplicado. El mismo de código muy similar puede incluirse en diferentes lugares de un programa. Éste se descarta o se implementa como un solo método o función que se llame cuando se requiera.

Métodos largos. Si un método es demasiado largo, debe rediseñarse en varios métodos más cortos. Enunciados de switch (case). Con frecuencia éstos implican duplicación, donde el cambio (switch) depende del tipo de algún valor. Los enunciados switch pueden dispersarse alrededor de un programa. En los lenguajes orientados a objetos, normalmente es posible usar un polimorfismo para lograr lo mismo.

Aglomeración de datos. Las aglomeraciones de datos ocurren cuando el mismo grupo de objetos de datos (campos en clases, parámetros en métodos) vuelven a ocurrir en muchos lugares en un programa. Generalmente pueden sustituirse con un objeto que encapsula todos los datos.

Generalidad especulativa. Esto ocurre cuando los desarrolladores incluyen generalidad en un programa, en caso de que se requiera en el futuro. Por lo general, esto simplemente puede eliminarse.

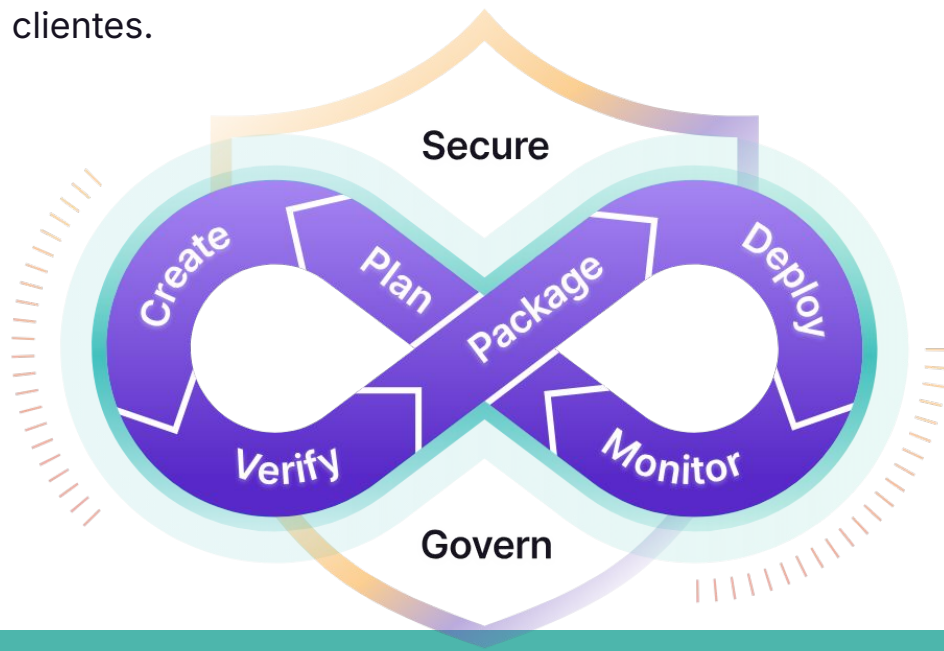
5 Min. de Pausa



DevOps

¿Qué es devops?

DevOps combina el desarrollo (Dev) y las operaciones (Ops) para aumentar la eficiencia, la velocidad y la seguridad del desarrollo y la entrega de software en comparación con los procesos tradicionales. Un ciclo de vida de desarrollo de software más ágil da como resultado una ventaja competitiva para las empresas y sus clientes.



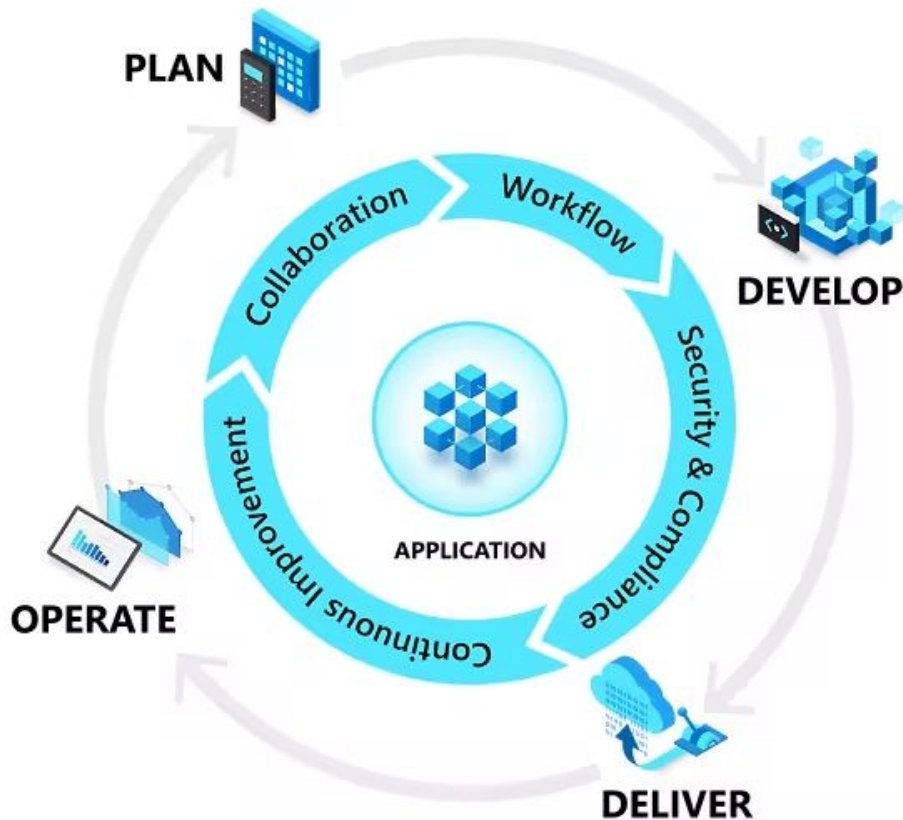
**WORKED FINE IN
DEV**

OPS PROBLEM NOW

¿Qué es devops?

Las personas que trabajan juntas para concebir, construir y entregar software seguro a la máxima velocidad.

Las prácticas de DevOps permiten a los equipos de desarrollo de software (dev) y operaciones (ops) acelerar la entrega a través de la automatización, la colaboración, la retroalimentación rápida y la mejora iterativa.



Principios básicos de DevOps

Automatización del ciclo de vida del desarrollo de software. Esto incluye la automatización de pruebas, compilaciones, versiones, el aprovisionamiento de entornos de desarrollo y otras tareas manuales que pueden ralentizar o introducir errores humanos en el proceso de entrega de software.

Colaboración y comunicación. Un buen equipo de DevOps tiene automatización, pero un gran equipo de DevOps también tiene una colaboración y comunicación efectivas.

Mejora continua y minimización de residuos. Desde la automatización de tareas repetitivas hasta la observación de las métricas de rendimiento en busca de formas de reducir los tiempos de lanzamiento o el tiempo medio de recuperación, los equipos de DevOps de alto rendimiento buscan regularmente áreas que podrían mejorarse.

Hiperenfoco en las necesidades del usuario con ciclos de retroalimentación cortos. A través de la automatización, la mejora de la comunicación y la colaboración, y la mejora continua, los equipos de DevOps pueden tomarse un momento y centrarse en lo que realmente quieren los usuarios reales y en cómo dárselo.

El ciclo de vida de DevOps

Plan: Organice el trabajo que debe realizarse, priorícelo y realice un seguimiento de su finalización.

Crear: Escriba, diseñe, desarrolle y administre de forma segura el código y los datos del proyecto con su equipo.

Verificar: Asegúrese de que su código funcione correctamente y se adhiera a sus estándares de calidad, idealmente con pruebas automatizadas.

Empaquetar: Empaquete sus aplicaciones y dependencias, administre contenedores y cree artefactos.

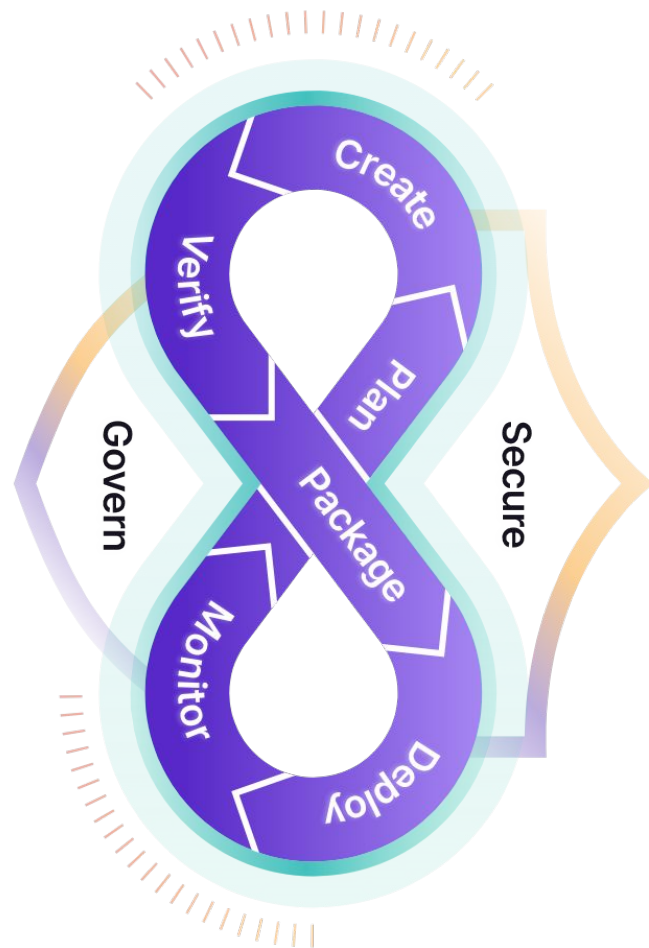
Asegurar: Compruebe si hay vulnerabilidades a través de pruebas estáticas y dinámicas, pruebas de fuzz y análisis de dependencias.

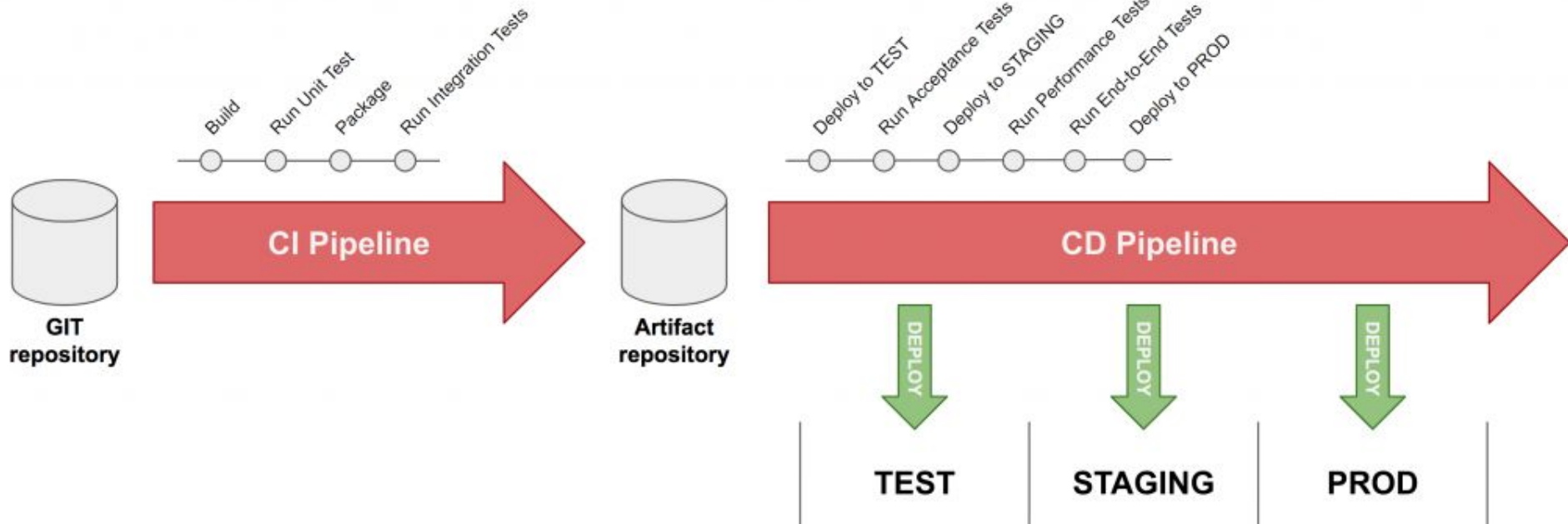
Lanzar: Implemente el software para los usuarios finales.

Configurar: Administre y configure la infraestructura necesaria para dar soporte a sus aplicaciones.

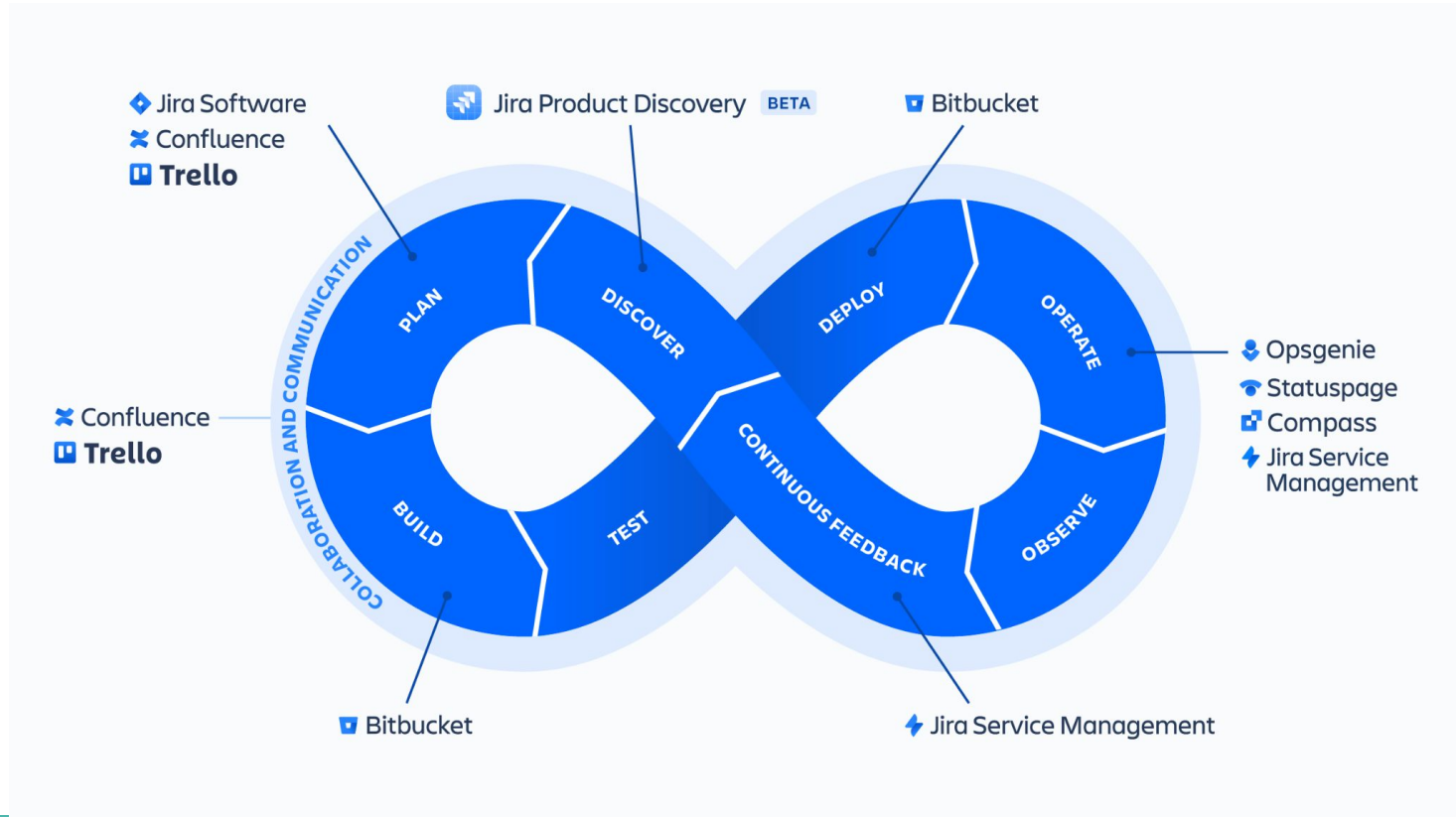
Monitorizar: Realice un seguimiento de las métricas de rendimiento y los errores para ayudar a reducir la gravedad y la frecuencia de los incidentes.

Gobernar: administre las vulnerabilidades de seguridad, las políticas y el cumplimiento en toda la organización.



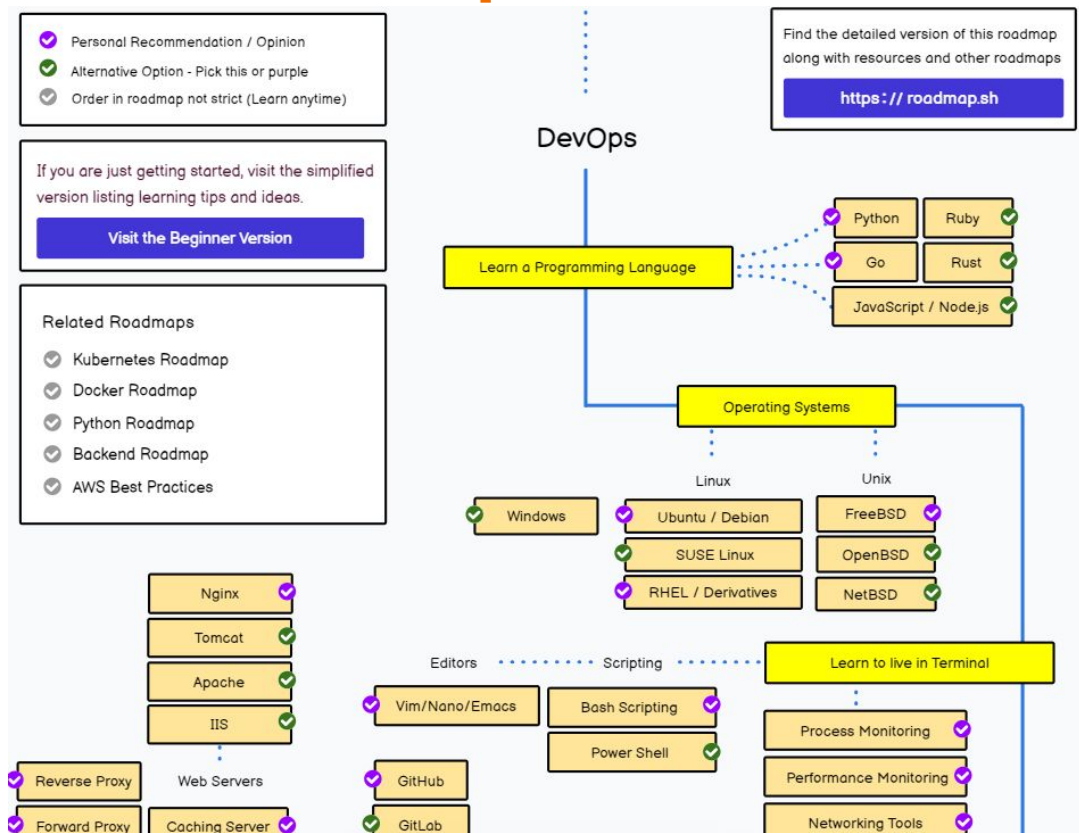


Suite de herramientas Atlassian



Roadmap para transformarte en DevOps

[DevOps Roadmap: Learn to become a DevOps Engineer or SRE](#)



Gestión del servicio - ITIL



¿Qué es ITIL?

ITIL es una biblioteca de mejores prácticas para gestionar servicios de TI y mejorar el soporte de TI y los niveles de servicio. Uno de los objetivos principales de ITIL es garantizar que los servicios de TI se alineen con los objetivos empresariales, incluso cuando estos cambien.

*ITIL corresponden a Information Technology Infrastructure Library (biblioteca de infraestructura de tecnologías de la información)

ITIL V4

ITIL service value chain	ITIL practices	ITIL guiding principles	Governance	Continual improvement
Plan	General Management Practices	Focus on value	Directs and controls the organization	Seven step improvement
Engage		Start where you are		
Design and Transition	Service Management Practices	Progress iteratively with feedback		
Obtain/Build	Technology Management Practices	Collaborate and promote visibility		
Deliver and Support		Think and work holistically		
Improve		Keep it simple and practical		
		Optimize and automate		

Cinco etapas clave



ITIL Process Diagrams

Problem Management



Incident Management



Change Management



Para profundizar

Capítulo 9 Sommerville